

Λειτουργικά Συστήματα Η/Υ

Κεφάλαιο 6 «Αδιέξοδο»

Διδάσκων: Δ. Λιαροκάπης
Διαφάνειες: Π. Χατζηδούκας

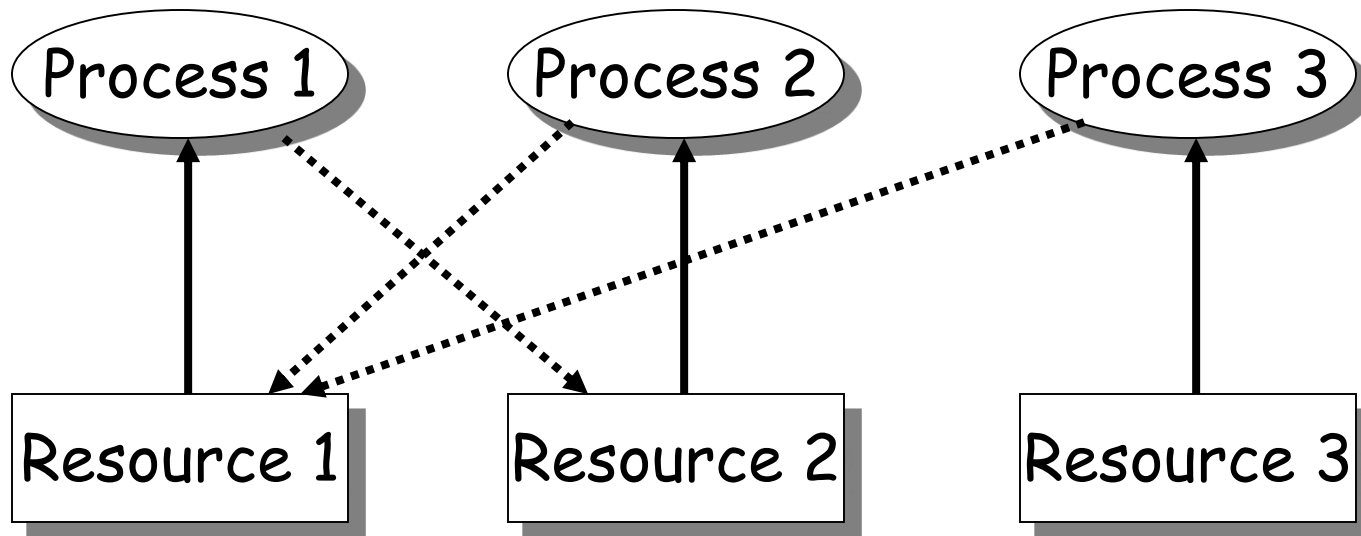
Αδιέξοδο

1. Ορισμοί – είδη πόρων
2. Γράφοι εκχώρησης πόρων
3. Συνθήκες αδιεξόδου
4. Προσεγγίσεις αδιεξόδου
 1. Πρόληψη
 2. Αποφυγή
 3. Ανίχνευση
5. Το πρόβλημα των συνδαιτημόνων φιλοσόφων

1. Εισαγωγή

- Αδιέξοδο (deadlock) είναι η μόνιμη ή επ' αόριστον αναμονή ενός συνόλου διεργασιών που είτε συναγωνίζονται για πόρους του συστήματος είτε επικοινωνούν μεταξύ τους.
- Σε ένα σύστημα πολυπρογραμματισμού η συνολική απαίτηση πόρων από όλες τις ενεργές διεργασίες υπερβαίνει κατά πολύ το συνολικό ποσό των διαθέσιμων πόρων.
- Όλα τα αδιέξοδα εμπεριέχουν τις συγκρουόμενες ανάγκες για πόρους από δύο ή περισσότερες διεργασίες
- Βασικός σκοπός είναι η σχεδίαση συστημάτων όπου το αδιέξοδο δεν θα μπορεί να συμβεί
- Γενικά δεν υπάρχει αποτελεσματική λύση

Παράδειγμα (1)



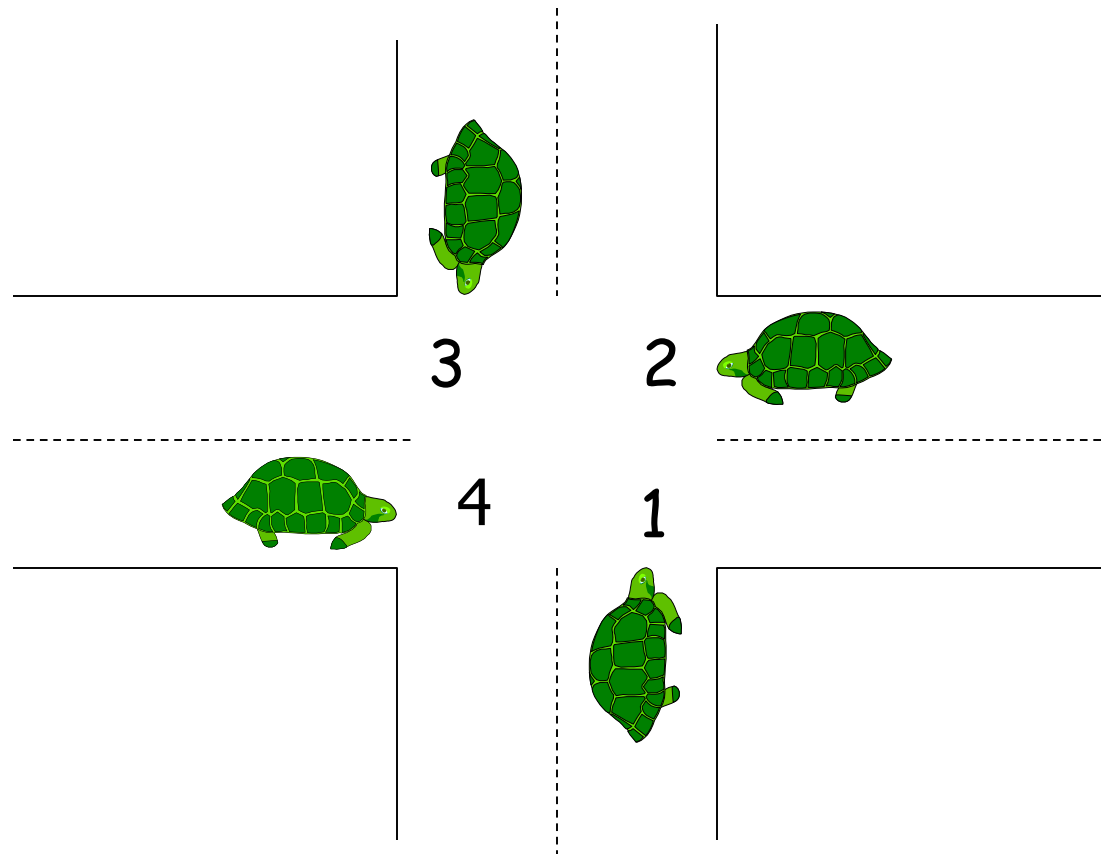
Πόροι (Resources)

- Προεκχωρούμενοι πόροι (preemptable resources)
 - Μπορούν να απομακρυνθούν από μια διεργασία χωρίς παρενέργειες
- Μη προεκχωρούμενοι πόροι (nonpreemptable resources)
 - Προξενούν αποτυχία στη διεργασία όταν απομακρυνθούν
 - Υπεύθυνοι για την εμφάνιση αδιεξόδων
- Παραδείγματα πόρων
 - εκτυπωτές, tape drives, μνήμη, CD - Recorders κλπ

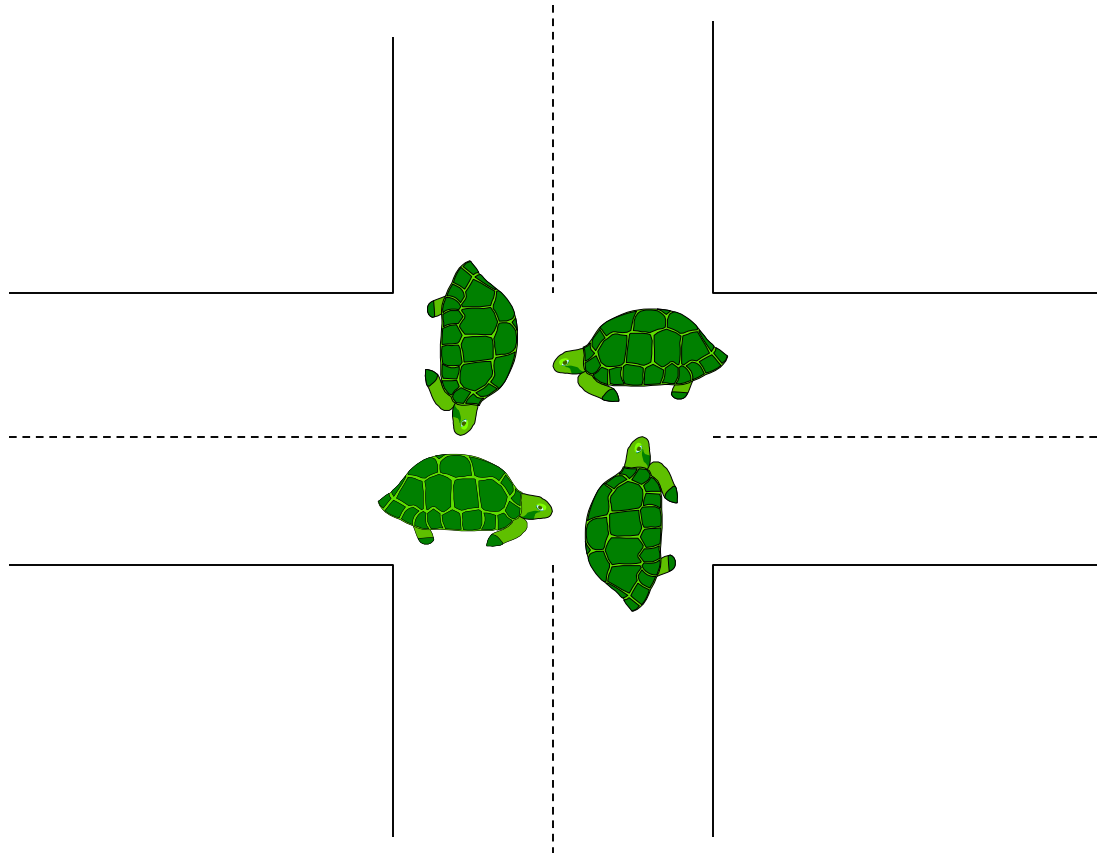
Χρήση πόρων

- Σειρά γεγονότων που απαιτούνται για τη χρήση ενός πόρου:
 - Απαίτηση (request)
 - Χρήση
 - Απελευθέρωση του πόρου
- Αναμονή αν η απαίτηση δεν ικανοποιηθεί
 - Η αιτούμενη διεργασία αναστέλλεται
 - Η αιτούμενη διεργασία αποτυγχάνει εμφανίζοντας μήνυμα λάθους

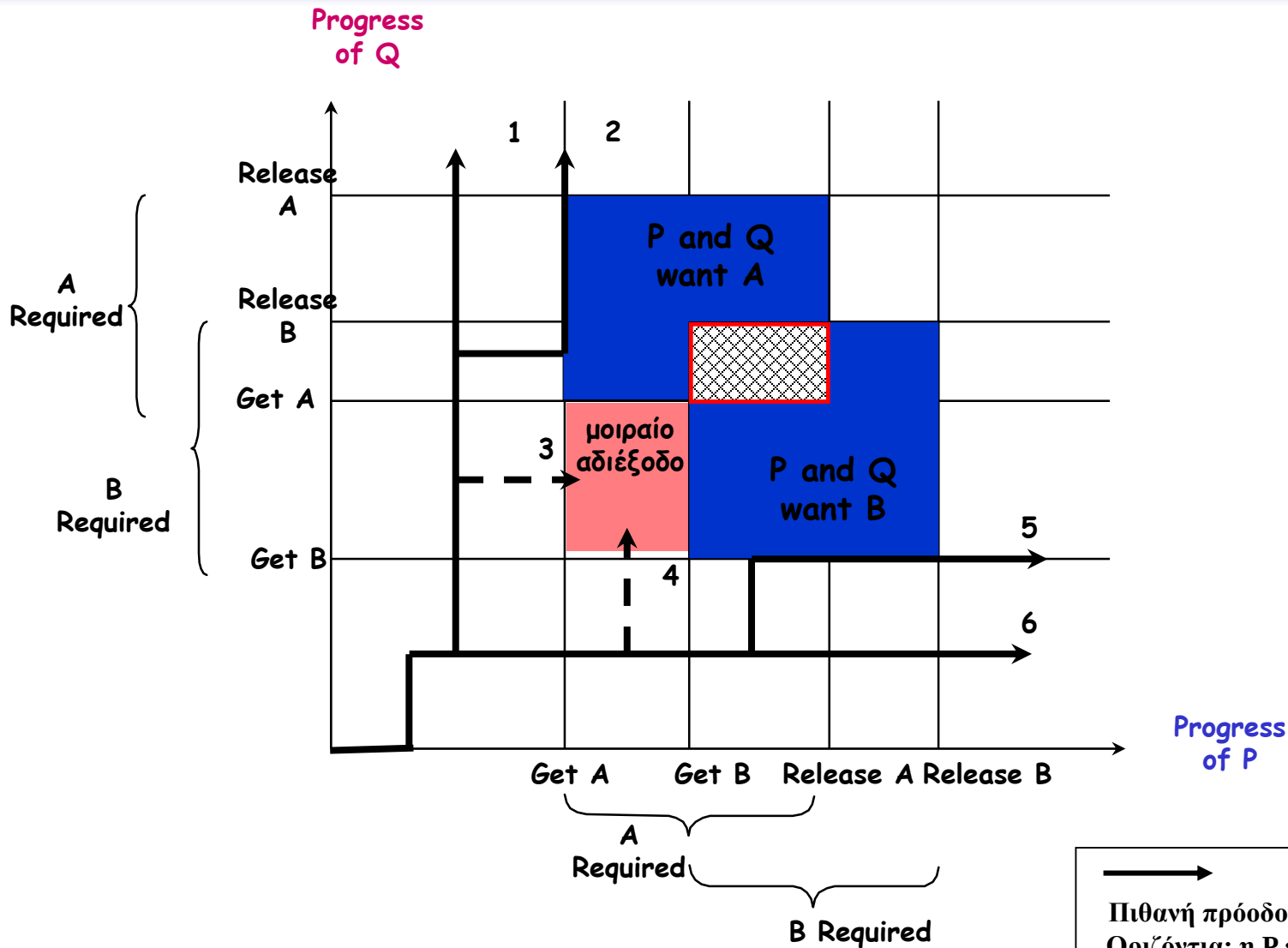
Πιθανό αδιέξοδο



Κατάσταση αδιεξόδου

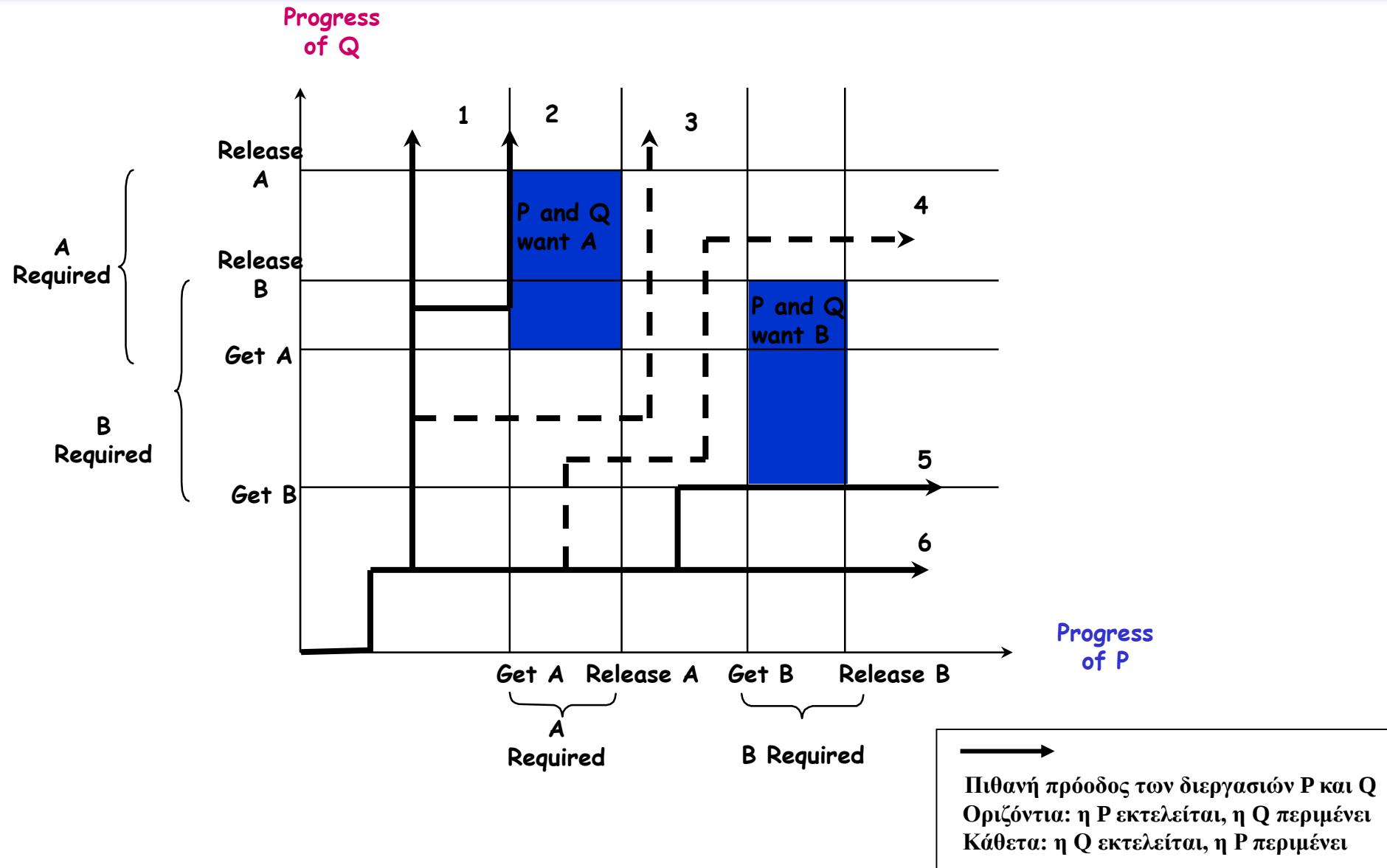


Παράδειγμα αδιεξόδου



→
Πιθανή πρόοδος των διεργασιών P και Q
Οριζόντια: η P εκτελείται, η Q περιμένει
Κάθετα: η Q εκτελείται, η P περιμένει

Παράδειγμα χωρίς αδιέξοδο



Επαναχρησιμοποιήσιμοι Πόροι

- Χρησιμοποιούνται με ασφάλεια από μια διεργασία σε κάθε χρονική στιγμή και δεν εξαντλούνται από αυτή τη χρήση.
- Οι διεργασίες αποκτούν πόρους που θα απελευθερώσουν στη συνέχεια ώστε να χρησιμοποιηθούν από άλλες διεργασίες.
- Τέτοιοι πόροι είναι: επεξεργαστές, I/O κανάλια, κύρια και δευτερεύουσα μνήμη, αρχεία, βάσεις δεδομένων, και σηματοφόροι.
- Το αδιέξοδο προκύπτει όταν μια διεργασία δεσμεύει ένα πόρο και απαιτεί έναν άλλο.

Παράδειγμα αδιεξόδου (1)

- Θεωρείστε τη σειρά εκτέλεσης: $p_0 p_1 q_0 q_1 p_2 q_2$

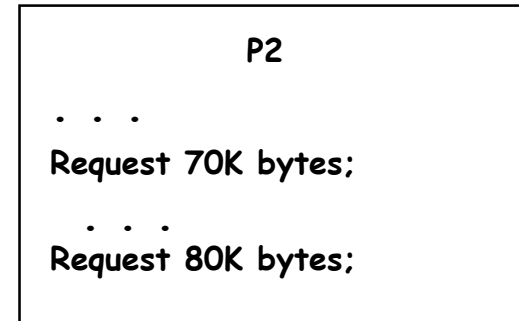
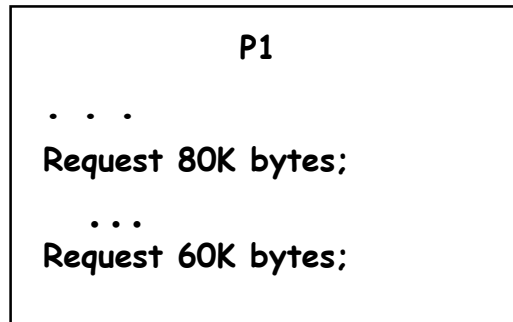
Process P		Process Q	
Step	Action	Step	Action
p_0	Request (D)	q_0	Request (T)
p_1	Lock (D)	q_1	Lock (T)
p_2	Request (T)	q_2	Request (D)
p_3	Lock (T)	q_3	Lock (D)
p_4	Perform function	q_4	Perform function
p_5	Unlock (D)	q_5	Unlock (T)
p_6	Unlock (T)	q_6	Unlock (D)

Figure 6.4 Example of Two Processes Competing for Reusable Resources

- Η αιτία αδιεξόδων αυτού του είδους είναι συχνά η πολύπλοκη λογική των προγραμμάτων.
- Μια στρατηγική αντιμετώπισης είναι η επιβολή περιορισμών που να αφορούν τη σειρά με την οποία ζητούνται οι πόροι.

Παράδειγμα αδιεξόδου (2)

- Ο διαθέσιμος χώρος για κατανομή στην κύρια μνήμη είναι 200Kbytes, και πραγματοποιείται η παρακάτω σειρά αιτημάτων



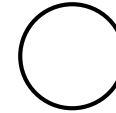
- Το αδιέξοδο προκύπτει αν και οι δύο διεργασίες προχωρήσουν στο δεύτερο αίτημά τους
- Η μνήμη είναι προεκχωρούμενος πόρος και το αδιέξοδο επιλύεται εύκολα (Πώς?)

Καταναλώσιμοι πόροι

- Δημιουργούνται (παράγονται) και καταστρέφονται (καταναλώνονται) από μια διεργασία
- Όταν ένας πόρος δεσμεύεται από μια διεργασία παύει να υπάρχει.
- Τέτοιοι πόροι: Διακοπές (Interrupts), σήματα (signals), μηνύματα και πληροφορίες σε ενταμιευτές (buffers) I/O
- Το αδιέξοδο μπορεί να συμβεί αν ένα μήνυμα που αποστέλλεται από μια διεργασία δεν παραλαμβάνεται από την άλλη
- Σπάνιος συνδυασμός γεγονότων μπορεί να οδηγήσει σε αδιέξοδο

2. Γράφοι εκχώρησης πόρων

- Διεργασία

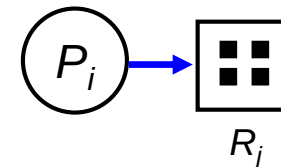


- Τύπος Πόρου με 4 στιγμιότυπα

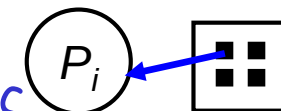


Σειρά
χρήσης
πόρου
μιας
διεργασίας

- P_i απαιτεί ένα στιγμιότυπο του R_j
Ακμή απαίτησης



- P_i δεσμεύει ένα στιγμιότυπο του R_j
Ακμή εκχώρησης

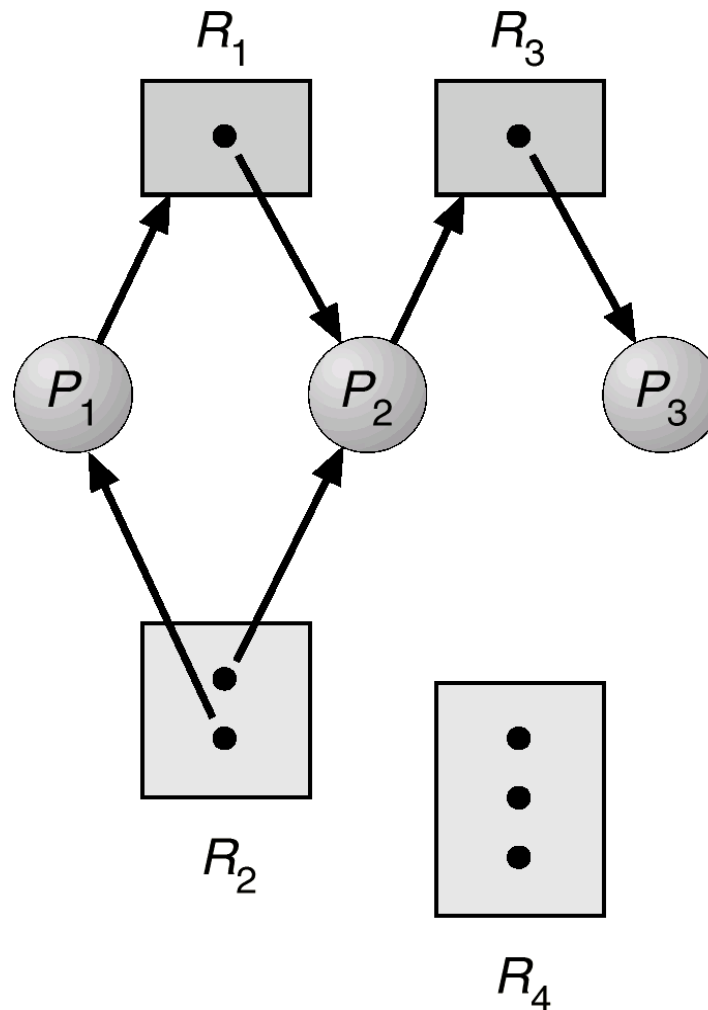


- P_i απελευθερώνει ένα στιγμιότυπο του R_j



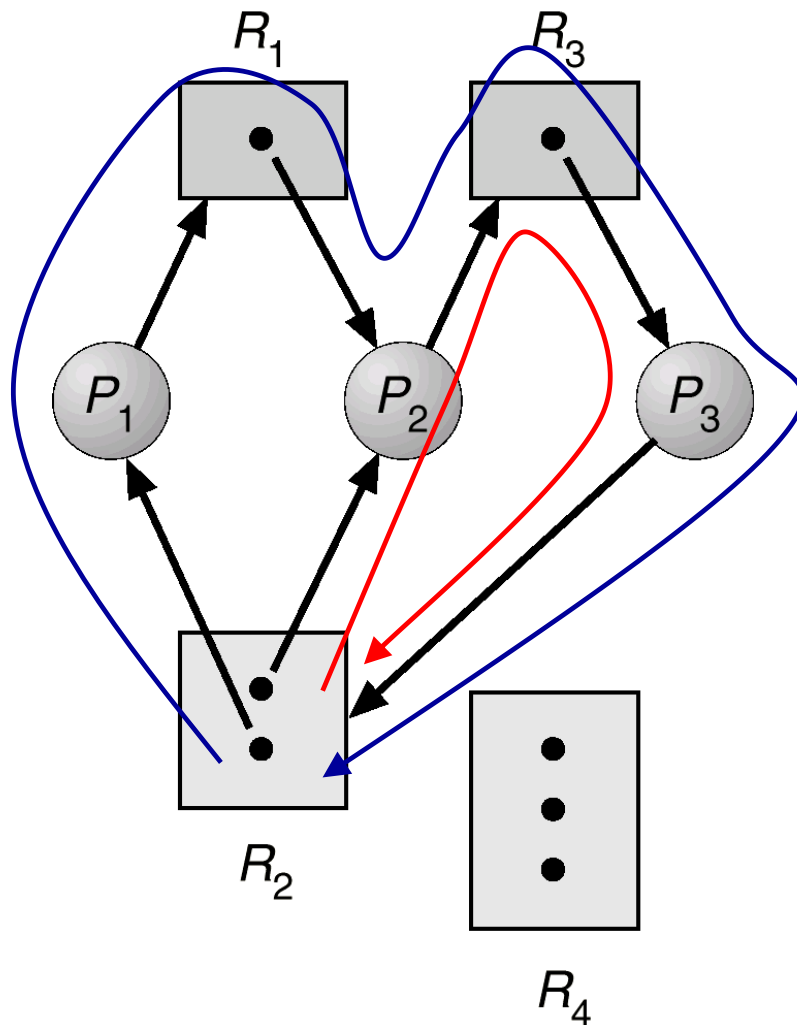
Παράδειγμα γράφου εκχώρησης πόρων

- Μπορεί να συμβεί αδιέξοδος;

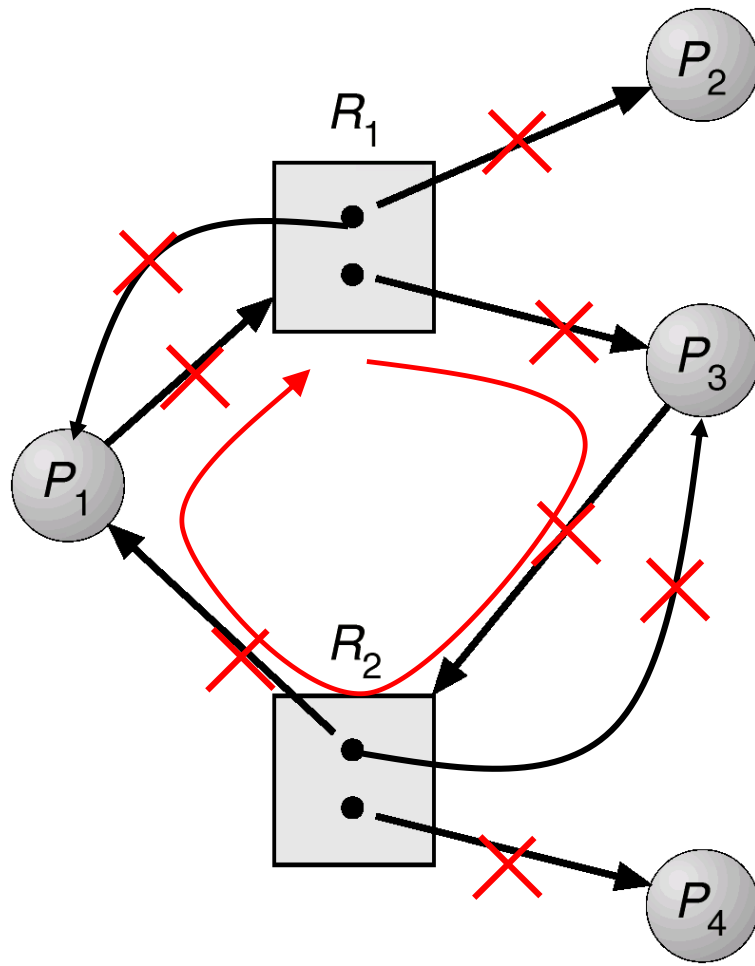


Παράδειγμα γράφου με αδιέξοδο

- Υπάρχουν δύο κύκλοι



Παράδειγμα γράφου εκχώρησης πόρου με κύκλο αλλά χωρίς αδιέξοδο



- Αν ο γράφος δεν περιέχει κύκλους $\Rightarrow$ δεν υπάρχει αδιέξοδο
- Αν ο γράφος περιέχει ένα κύκλο $\Rightarrow$
 - Αν υπάρχει μόνον ένα στιγμιότυπο ανά τύπο πόρου, τότε υπάρχει αδιέξοδο.
 - Αν υπάρχουν αρκετά στιγμιότυπα ανά τύπο πόρου, τότε **υπάρχει πιθανότητα να συμβεί αδιέξοδο.**

Γράφοι εκχώρησης και αδιέξοδο

- Αν ο γράφος δεν περιέχει κύκλους
 - δεν υπάρχει αδιέξοδο
- Αν ο γράφος περιέχει ένα κύκλο
 - Αν υπάρχει μόνον ένα στιγμιότυπο ανά τύπο πόρου, τότε υπάρχει αδιέξοδο.
 - Αν υπάρχουν αρκετά στιγμιότυπα ανά τύπο πόρου, τότε υπάρχει πιθανότητα να συμβεί αδιέξοδο.

Άσκηση 1

- Να σχεδιάσετε ένα γράφο εκχώρησης πόρων για τα παρακάτω:
 - Η διεργασία P1 απαιτεί τον πόρο R1
 - Η διεργασία P2 απαιτεί τον πόρο R3
 - Ο πόρος R1 εκχωρείται στη διεργασία P2
 - Ο πόρος R2 εκχωρείται στη διεργασία P1
 - Ο πόρος R3 εκχωρείται στη διεργασία P1
- Υπάρχει αδιέξοδο;

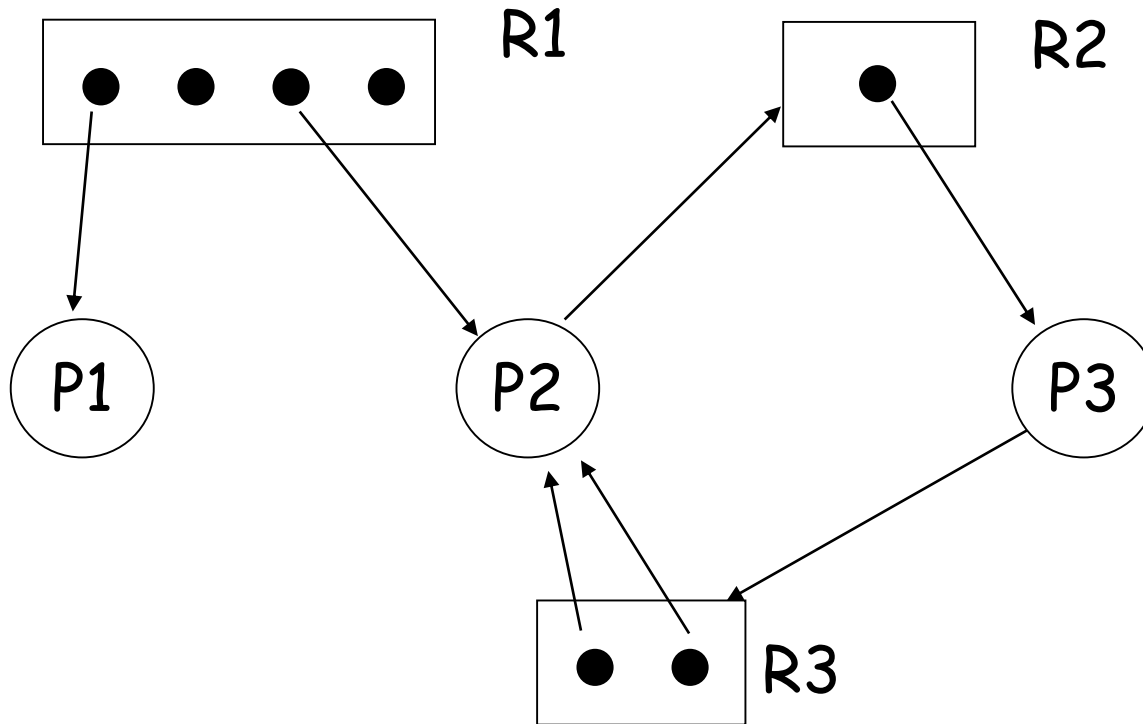
Άσκηση 2

- Ένα σύστημα διαθέτει 6 όμοια tape drives και n σε πλήθος διεργασίες που ανταγωνίζονται για τη χρήση τους. Κάθε διεργασία μπορεί να απαιτήσει 2 tape drives.
- Για ποια τιμή του n το σύστημα είναι απαλλαγμένο από αδιέξοδα;

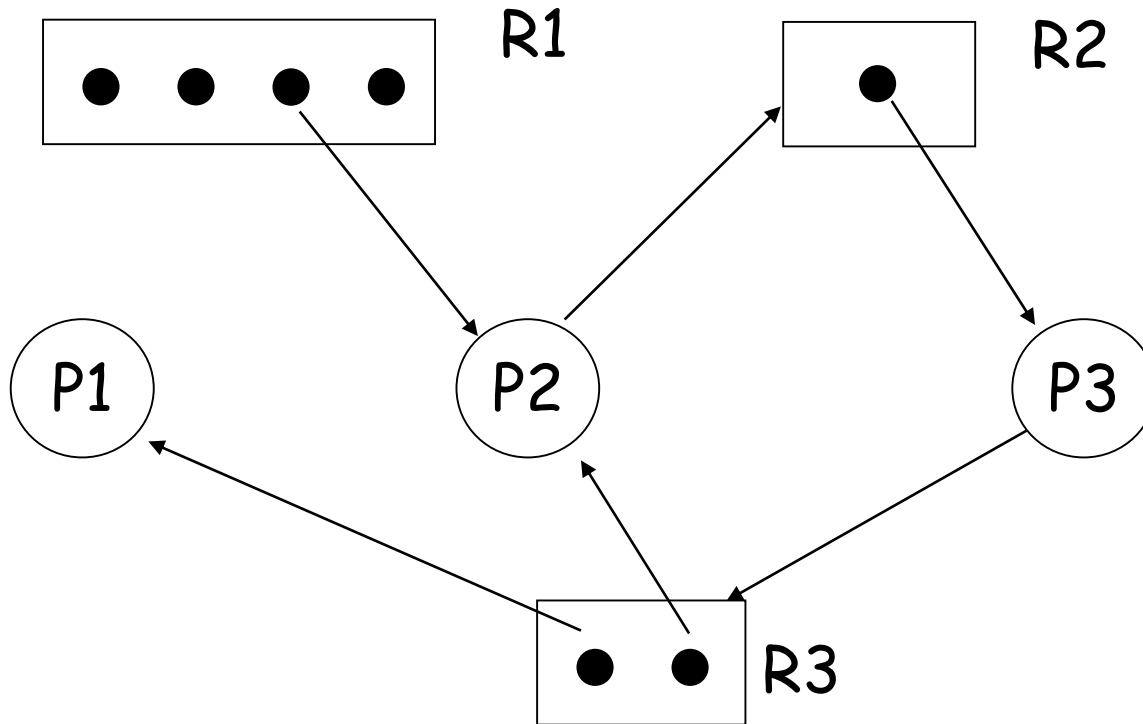
3. Συνθήκες αδιεξόδου

- Το αδιέξοδο συμβαίνει όταν ισχύουν όλα τα παρακάτω (συνθήκες αδιεξόδου):
 - Αμοιβαίος αποκλεισμός: οι εκχωρούμενοι πόροι είναι στην αποκλειστική κυριότητα της διεργασίας. Κάθε πόρος εκχωρείται σε μία διεργασία ή είναι διαθέσιμος.
 - Κατοχή και αναμονή: Η διεργασία μπορεί να δεσμεύει έναν πόρο ενώ περιμένει έναν άλλο. Η διεργασία μπορεί να έχει δεσμεύσει πόρους και στη συνέχεια να ζητήσει και άλλους.
 - Μη προεκχώρηση: Κανένας πόρος δεν μπορεί να αποσπασθεί δια της βίας από τη διεργασία που την κατέχει.
 - Κυκλική αναμονή: Ύπαρξη μιας κλειστής αλυσίδας διεργασιών 2 ή περισσότερων διεργασιών. Κάθε μία αναμένει ένα πόρο που κατέχεται από το επόμενο μέλος της αλυσίδας.

Παράδειγμα κυκλικής αναμονής



Παράδειγμα χωρίς κυκλική αναμονή



4. Προσεγγίσεις αδιεξόδου

- Το αδιέξοδο είναι μια γενική κατάσταση
- Δεν υπάρχει μια ενιαία στρατηγική αντιμετώπισης κάθε είδους αδιεξόδου.
- Απαιτεί την ανάλυση όλων των διεργασιών που χρειάζονται πόρους.
- Δεν μπορεί να ληφθεί μια τοπική απόφαση που θα στηρίζεται στις ανάγκες μιας διεργασίας
- Υπάρχουν 4 προσεγγίσεις αδιεξόδου:

Προσεγγίσεις αδιεξόδου

1. Πρόληψη (Prevention): Δεν επιτρέπεται ποτέ να συμβεί αδιέξοδο
2. Αποφυγή (Avoidance): Το σύστημα λαμβάνει απόφαση για να αποτρέψει μελλοντική κατάσταση αδιεξόδου
3. Ανίχνευση (Detection) & Επαναφορά (Recovery) : έλεγχος για αδιέξοδο (περιοδικά ή σποραδικά), στη συνέχεια επαναφορά
4. Χειροκίνητη μεσολάβηση: Ο χειριστής κάνει επανεκκίνηση του συστήματος, αν φαίνεται υπερβολικά αργό.

4.1. Πρόληψη αδιεξόδου

- Σχεδιασμός του συστήματος έτσι ώστε να παραβιάζει μία από τις 4 αναγκαίες συνθήκες για την εμφάνιση αδιεξόδου.
 - Αμοιβαίος αποκλεισμός
 - Κατοχή και αναμονή
 - Μη προεκχώρηση
 - Κυκλική αναμονή

Αμοιβαίος αποκλεισμός

- Μόνον μια διεργασία τη φορά μπορεί να χρησιμοποιεί ένα πόρο
- Λύση
 - Εξασφάλισε ότι όσο το δυνατόν λιγότερες διεργασίες πρόκειται να διεκδικήσουν τον πόρο στην πράξη
 - Παράδειγμα: εκτυπωτής – μόνον ο printer daemon μπορεί να χρησιμοποιεί τον πόρο (χρήση παροχέτευσης, κίνδυνος αδιεξόδου στην παροχέτευση)
 - Δεν μπορούν όλες οι συσκευές να λειτουργήσουν παρόμοια

Κατοχή και αναμονή (Hold-and-wait)

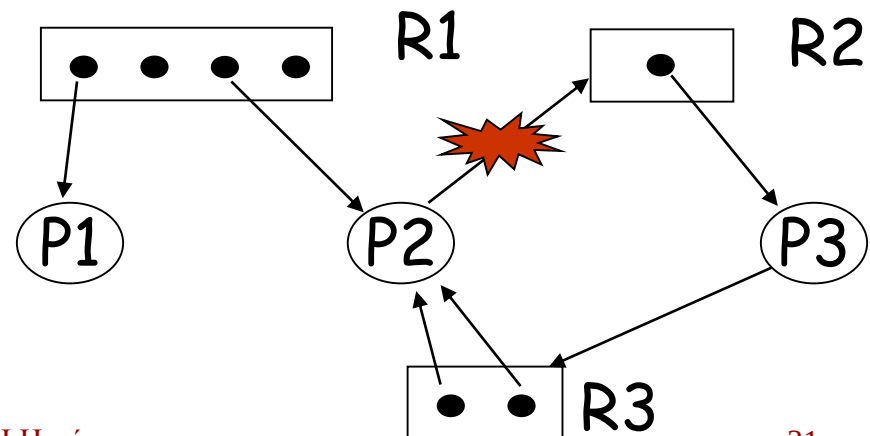
- Μια διεργασία μπορεί να κατέχει πόρους καθώς αναμένει την εκχώρηση κάποιων άλλων πόρων
- Λύση
 - Μια διεργασία απαιτεί όλους τους πόρους πριν ξεκινήσει (άρα δεν θα περιμένει ποτέ στη συνέχεια). Αν δεν είναι διαθέσιμοι αναμένει
 - Εναλλακτικά, αν ζητηθούν επιπλέον πόροι πρέπει πρώτα να αποδεσμευθούν οι ήδη δεσμευμένοι και να γίνει προσπάθεια δέσμευσης όλων των πόρων εξ αρχής
- Προβλήματα
 - Δεν είναι γνωστές οι απαιτήσεις πόρων κατά την έναρξη της διεργασίας
 - Παρατεταμένη στέρηση
 - Δεσμεύονται πόροι που θα μπορούσαν να χρησιμοποιηθούν από άλλες διεργασίες

Μη προεκχώρηση (No preemption)

- Κανένας πόρος δεν μπορεί να αποσπασθεί δια της βίας από τη διεργασία που τον κατέχει.
- Λύση
 - Αφαιρούμε προσωρινά έναν πόρο από τη διεργασία που τον κατέχει και τον παραχωρούμε σε κάποια άλλη διεργασία
 - Εφαρμόζεται σε πόρους η κατάσταση των οποίων μπορεί να αποθηκευθεί και αργότερα να γίνει επαναφορά, όπως : καταχωρητές της CPU και χώρος μνήμης
 - Δεν είναι μια βιώσιμη επιλογή για την πλειονότητα των πόρων
 - Παράδειγμα
 - Η διακοπή μιας εγγραφής CD δημιουργεί ένα σημαντικό κόστος
 - Ο οδηγός συσκευής του CD-R απαγορεύει μια δεύτερη λειτουργία open()

Κυκλική αναμονή

- Ύπαρξη κυκλικής αλυσίδα με 2 ή περισσότερες διεργασίες όπου κάθε μια διεργασία περιμένει έναν πόρο που κατέχεται από άλλη διεργασία της αλυσίδας
- Λύση
 - Ορισμός μια γραμμικής αρίθμησης των πόρων
 - Οι αιτήσεις για πόρους από τις διεργασίες πρέπει να γίνονται με βάση τη σειρά αρίθμησης
 - Η ποικιλομορφία των πόρων και των χρήσεών τους κάνει δύσκολη την εφαρμογή στην πράξη



4.2. Αποφυγή αδιεξόδου

- Οι πόροι εκχωρούνται με τρόπο που εγγυάται ότι δεν θα βρεθεί ποτέ σημείο στο οποίο θα συμβεί αδιέξοδο
- Η αποφυγή του αδιεξόδου επιτρέπει τις τρεις πρώτες απαραίτητες συνθήκες αλλά κάνει διακριτικές επιλογές ώστε να εξασφαλιστεί ότι δεν θα προκύψει ποτέ αδιέξοδο. Έτσι επιτρέπεται μεγαλύτερος συγχρονισμός σε σχέση με την πρόληψη.
- **Απαιτείται γνώση των μελλοντικών απαιτήσεων της διεργασίας.**
- Προϋποθέτει ότι το σύστημα έχει κάποια πρόσθετη και εκ των προτέρων διαθέσιμη πληροφόρηση.

Αποφυγή αδιεξόδου

- Λαμβάνεται δυναμικά μια απόφαση σχετικά με το αν η ικανοποίηση μιας απαίτησης , σε σχέση με την τρέχουσα εκχώρηση πόρων, θα οδηγήσει σε αδιέξοδο, που βασίζεται σε :
 - Στο συνολικό ποσό των διαθέσιμων πόρων
 - Στους τρέχοντες διαθέσιμους πόρους
 - Στις απαιτήσεις πόρων εκ μέρους των διεργασιών
 - Στην πρόσφατη εκχώρηση πόρων στις διεργασίες

Αποφυγή αδιεξόδου

- Το απλούστερο και πλέον χρήσιμο μοντέλο απαιτεί ότι κάθε διεργασία δηλώνει το μέγιστο πλήθος των πόρων κάθε τύπου που είναι πιθανόν να χρειαστεί.
- Ο αλγόριθμος αποφυγής αδιεξόδου εξετάζει δυναμικά την κατάσταση ανάθεσης πόρων για να εξασφαλίσει ότι δεν μπορεί να προκύψει κατάσταση κυκλικής αναμονής.
- Η κατάσταση ανάθεσης πόρων ορίζεται από το πλήθος των διαθέσιμων και εκχωρούμενων πόρων και από το μέγιστο πλήθος των απαιτήσεων εκ μέρους των διεργασιών.

Αποφυγή αδιεξόδου

- Προσεγγίσεις αποφυγής αδιεξόδου
 - Μια διεργασία δεν ξεκινά αν οι απαιτήσεις της μπορούν να οδηγήσουν σε αδιέξοδο
 - Δεν ικανοποιείται μια αυξημένη απαίτηση για τη χρήση ενός πόρου από μια διεργασία αν αυτή η εκχώρηση του πόρου μπορεί να οδηγήσει σε αδιέξοδο
 - Προβλήματα
 - Μικρή χρήση πόρων
 - Μειωμένη ρυθμοαπόδοση (throughput) του συστήματος

Αλγόριθμος του τραπεζίτη

- Είναι γνωστός και ως άρνηση ανάθεσης πόρων
- Δεν επιτρέπει την ικανοποίηση αυξημένων απαιτήσεων για πόρους σε μια διεργασία αν αυτή η εκχώρηση πόρων μπορεί να οδηγήσει σε αδιέξοδο.
- Κατάσταση του συστήματος: είναι η τρέχουσα ανάθεση πόρων στις διεργασίες
- Ασφαλής κατάσταση: υπάρχει μια τουλάχιστον ακολουθία εκτέλεσης των διεργασιών που μπορεί να εκτελεστεί μέχρι το τέλος (δηλαδή δεν οδηγεί σε αδιέξοδο)
- Μη ασφαλής κατάσταση: είναι η κατάσταση που δεν είναι ασφαλής

Ασφαλής κατάσταση

- Όταν μια διεργασία απαιτεί ένα διαθέσιμο πόρο, το σύστημα πρέπει να αποφασίσει αν η άμεση ανάθεση του πόρου θα το διατηρήσει σε ασφαλή κατάσταση.
- Το σύστημα είναι σε ασφαλή κατάσταση αν υπάρχει μια ασφαλής ακολουθία για όλες τις διεργασίες.
- Η ακολουθία $\langle P_1, P_2, \dots, P_n \rangle$ είναι ασφαλής αν για κάθε P_i , οι πόροι που η P_i μπορεί ακόμη να απαιτήσει μπορούν να ικανοποιηθούν από τους τρέχοντες διαθέσιμους πόρους συν τους πόρους που δεσμεύονται από όλες τις διεργασίες P_j , με $j < i$.

Ασφαλής κατάσταση

- Αν οι ανάγκες σε πόρους της P_i δεν είναι άμεσα διαθέσιμοι, τότε η P_i μπορεί να περιμένει μέχρις ότου να τελειώσουν όλες οι P_j .
- Όταν τελειώνει η P_j η P_i μπορεί να αποκτήσει τους πόρους που χρειάζεται, να εκτελεστεί, να επιστρέψει τους πόρους που της ανατέθηκαν και να τερματιστεί.
- Όταν τερματίσει η P_i , η P_{i+1} μπορεί να αποκτήσει τους πόρους που χρειάζεται κ.ο.κ.

Βασικά γεγονότα

- Αν το σύστημα είναι σε ασφαλή κατάσταση δεν υπάρχει αδιέξοδο.
- Αν το σύστημα δεν είναι σε ασφαλή κατάσταση υπάρχει πιθανότητα αδιεξόδου (το αδιέξοδο δεν θα συμβεί σίγουρα, απλώς είναι πιθανό)
- Αποφυγή αδιεξόδου : εξασφάλιση ότι το σύστημα δεν θα εισέλθει ποτέ σε μη ασφαλή κατάσταση

Παραδοχές

- Πολλαπλά στιγμιότυπα των πόρων
- Κάθε διεργασία πρέπει εκ των προτέρων να διεκδικεί τη μέγιστη χρήση των πόρων
- Όταν μια διεργασία απαιτεί έναν πόρο ίσως χρειαστεί να περιμένει
- Όταν μια διεργασία λάβει όλους τους πόρους πρέπει να τους επιστρέψει σε πεπερασμένο χρονικό διάστημα.
- Η μέγιστη απαίτηση για πόρους πρέπει να δηλώνεται εκ των προτέρων
- Οι σημαντικές διεργασίες πρέπει να είναι ανεξάρτητες και δεν υπόκεινται σε απαιτήσεις συγχρονισμού
- Πρέπει να υπάρχει ένας σταθερός αριθμός πόρων προς ανάθεση
- Καμιά διεργασία δεν μπορεί να περιέλθει σε κατάσταση εξόδου (exit) ενώ δεσμεύει πόρους

Δομές δεδομένων του αλγορίθμου

- n = το πλήθος των διεργασιών,
- m = το πλήθος τύπων πόρων
- Available: Διάνυσμα μήκους m . Αν $available[j] = k$, υπάρχουν k στιγμιότυπα του τύπου πόρου R_j διαθέσιμα.
- Max: $n \times m$ πίνακας. Αν $Max[i,j] = k$, τότε η διεργασία P_i μπορεί να απαιτήσει κατά μέγιστο k στιγμιότυπα του τύπου πόρου R_j .
- Allocation: $n \times m$ πίνακας. Αν $Allocation[i,j] = k$ τότε στη διεργασία P_i εκχωρούνται k στιγμιότυπα του πόρου R_j .
- Need: $n \times m$ πίνακας. Αν $Need[i,j] = k$, τότε η διεργασία P_i μπορεί να χρειαστεί k επιπλέον στιγμιότυπα του πόρου R_j για να ολοκληρωθεί.
- $Need[i,j] = Max[i,j] - Allocation[i,j]$

Αλγόριθμος

- Ψάξε στον πίνακα Need για μια γραμμή i όπου όλες οι απαιτήσεις είναι μικρότερες ή ίσες από τους διαθέσιμους πόρους, δλδ. $Need[i,j] \leq Available[j] \ \forall j$
 - Αν δεν υπάρχει τέτοια γραμμή τότε τερμάτισε. Η κατάσταση είναι **ανασφαλής**.
- Υπέθεσε ότι η διεργασία της γραμμής i ζητά όλους τους πόρους και τερματίζει.
 - Σημείωσε τη διεργασία ως ολοκληρωμένη.
 - Πρόσθεσε τους πόρους που κατείχε στους διαθέσιμους, δλδ. $Available[j] = Available[j] + Allocation[i,j] \ \forall j$
- Επανέλαβε τα παραπάνω μέχρι να σημειωθούν όλες οι διεργασίες ως ολοκληρωμένες.
 - Σε αυτή την περίπτωση η κατάσταση είναι **ασφαλής**.

Παράδειγμα

- Εφαρμογή του αλγορίθμου για καθορισμό μιας ασφαλούς κατάστασης
- Υπάρχουν 3 τύποι πόρων με πλήθος :
 - $R(1) = 9, R(2) = 3, R(3) = 6$
- Είναι η παρακάτω κατάσταση ασφαλής;

	Max	Allocated	Total	Need
	R1 R2 R3	R1 R2 R3	R1 R2 R3	R1 R2 R3
P1	3 2 2	1 0 0	9 3 6	2 2 2
<u>P2</u>	6 1 3	6 1 2		0 0 1
P3	3 1 4	2 1 1		1 0 3
P4	4 2 2	0 0 2		4 2 0
			Available 0 1 1	

Παράδειγμα

	Max			Allocated			Total			Need		
	R1	R2	R3	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	3	2	2	1	0	0	9	3	6	2	2	2
P2	0	0	0	0	0	0				0	0	0
P3	3	1	4	2	1	1	Available			1	0	3
P4	4	2	2	0	0	2				4	2	0

	Max			Allocated			Total			Need		
	R1	R2	R3	R1	R2	R3	R1	R2	R3	R1	R2	R3
<u>P1</u>	0	0	0	0	0	0	9	3	6	0	0	0
P2	0	0	0	0	0	0				0	0	0
P3	3	1	4	2	1	1	Available			1	0	3
P4	4	2	2	0	0	2				4	2	0

Παράδειγμα

	Max			Allocated			Total			Need		
	R1	R2	R3	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	0	0	0	0	0	0	9	3	6	0	0	0
P2	0	0	0	0	0	0				0	0	0
<u>P3</u>	0	0	0	0	0	0	Available			0	0	0
P4	4	2	2	0	0	2				4	2	0

	Max			Allocated			Total			Need		
	R1	R2	R3	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	0	0	0	0	0	0	9	3	6	0	0	0
P2	0	0	0	0	0	0				0	0	0
P3	0	0	0	0	0	0	Available			0	0	0
<u>P4</u>	0	0	0	0	0	0				0	0	0

Η κατάσταση είναι ασφαλής: $P2 \rightarrow P1 \rightarrow P3 \rightarrow P4$

4.3. Ανίχνευση αδιεξόδου

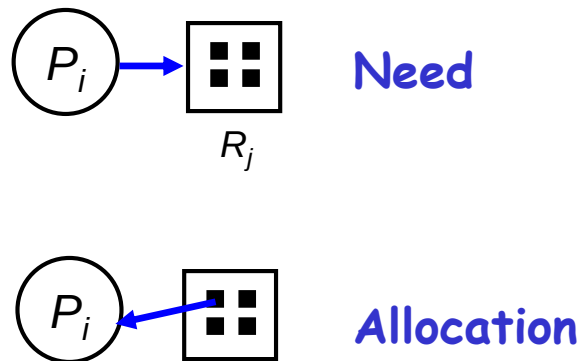
- Ο αλγόριθμος του τραπεζίτη είναι απαισιόδοξος : πάντοτε υποθέτει ότι μια διεργασία δεν θα απελευθερώσει τους πόρους που κατέχει μέχρι να αποκτήσει όλους τους πόρους που χρειάζεται.
 - Συνέπειες :
 - Μικρό ποσοστό παραλληλίας
 - Πολύπλοκοι έλεγχοι για κάθε απαίτηση εκχώρησης πόρου (πολυπλοκότητα $O(n^2)$)
- Οι στρατηγικές ανίχνευσης αδιεξόδου δεν οριοθετούν την πρόσβαση σε πόρους και δεν περιορίζουν τις ενέργειες των διεργασιών.
- Οι απαιτούμενοι πόροι εκχωρούνται στις διεργασίες, όποτε αυτό είναι δυνατό.

ΛΣ και ανίχνευση αδιεξόδου

- Περιοδικά το ΛΣ εφαρμόζει έναν αλγόριθμο ανίχνευσης αδιεξόδου που δίνει τη δυνατότητα να εντοπισθεί η συνθήκη της κυκλικής αναμονής στο γράφο εκχώρησης πόρων.
- Το πότε και πόσο συχνά καλείται ο αλγόριθμος εξαρτάται από:
- Πόσο συχνά συνηθίζεται να συμβαίνει αδιέξοδο
- Το πλήθος των διεργασιών που πρέπει να επιστρέψουν σε προηγούμενη κατάσταση

Αλγόριθμος ανίχνευσης αδιεξόδου

- n = το πλήθος των διεργασιών,
- m = το πλήθος τύπων πόρων
- Available: Διάνυσμα μήκους m . Αν $available[j] = k$, υπάρχουν k στιγμιότυπα του τύπου πόρου R_j διαθέσιμα.
- Allocation: $n \times m$ πίνακας. Αν $Allocation[i, j] = k$ τότε στη διεργασία P_i εκχωρούνται k στιγμιότυπα του πόρου R_j .
- Need: $n \times m$ πίνακας. Αν $Need[i, j] = k$, τότε η διεργασία P_i απαιτεί k επιπλέον στιγμιότυπα του πόρου R_j για να ολοκληρωθεί.



Αλγόριθμος

- Ψάξε στον πίνακα Need για μια γραμμή i όπου όλες οι απαιτήσεις είναι μικρότερες ή ίσες από τους διαθέσιμους πόρους, δλδ. $\text{Need}[i,j] \leq \text{Available}[j] \ \forall j$
 - Αν δεν υπάρχει τέτοια γραμμή τότε τερμάτισε. Οι διεργασίες που δεν έχουν σημειωθεί βρίσκονται σε **αδιέξοδο**.
- Η διεργασία της γραμμής i αποκτά όλους τους πόρους που απαιτεί και τερματίζει.
 - Σημείωσε τη διεργασία ως ολοκληρωμένη.
 - Πρόσθεσε τους πόρους που κατείχε στους διαθέσιμους, δλδ. $\text{Available}[j] = \text{Available}[j] + \text{Allocation}[i,j] \ \forall j$
- Επανέλαβε τα παραπάνω μέχρι να σημειωθούν όλες οι διεργασίες ως ολοκληρωμένες.
 - Σε αυτή την περίπτωση **δεν υπάρχει αδιέξοδο κατά την τρέχουσα χρονική στιγμή**.

ΛΣ και ανίχνευση αδιεξόδου

- Στην πράξη, τα περισσότερα Λ.Σ. εθελουφλούν και αντιμετωπίζουν το πρόβλημα με συνδυασμό τεχνικών, όπως:
 - Διακοπή κατοχής και αναμονής:
 - Όταν μια διεργασία δεν μπορεί να αποκτήσει έναν πόρο, τότε δεν μπορεί να ολοκληρωθεί και αποτυγχάνει
 - Όρια (Quotas)
 - Υψηλής απόδοσης τεχνικές προγραμματισμού:
 - Απαίτηση χρήσης σημαφόρων με καθορισμένη προτεραιότητα.

Στρατηγικές όταν ανιχνευθεί αδιέξοδο

- Τερματισμός διεργασίας & προεκχώρηση πόρων
 - Διακοπή όλων των διεργασιών που περιήλθαν σε αδιέξοδο
 - Διαδοχική διακοπή όλων των διεργασιών που βρίσκονται σε αδιέξοδο ώστε να μην υπάρχει πλέον αδιέξοδο
 - Δημιουργία αντιγράφου ασφαλείας για κάθε διεργασία μέσω περιοδικών σημείων ελέγχου. Οι διεργασίες που βρίσκονται σε αδιέξοδο επιστρέφουν σε κάποιο προηγούμενο σημείο ελέγχου και συνεχίζουν από εκεί.
 - Οι πόροι που οδήγησαν στο αδιέξοδο αποδεσμεύονται
 - Το αρχικό αδιέξοδο μπορεί να ξανασυμβεί
 - Διαδοχική προεκχώρηση πόρων έως ότου δεν θα υπάρχει αδιέξοδο

Στρατηγικές όταν ανιχνευθεί αδιέξοδο

- Κριτήρια επιλογής των διεργασιών που βρίσκονται σε αδιέξοδο και θα τερματιστούν
 - Επιλέγεται η διεργασία που έχει :
 - καταναλώσει το μικρότερο ποσό χρόνου επεξεργασίας μέχρι την τρέχουσα στιγμή
 - παράγει το μικρότερο πλήθος γραμμών εξόδου ποσό μέχρι την τρέχουσα στιγμή
 - τον μεγαλύτερο εκτιμώμενο χρόνο
 - το μικρότερο πλήθος πόρων που της έχουν εκχωρηθεί
 - τη μικρότερη προτεραιότητα

Συνδυασμένη ανίχνευση αδιεξόδου

- Ο συνδυασμός των τριών προσεγγίσεων
 - Πρόληψης,
 - Αποφυγής,
 - Ανίχνευσηςεπιτρέπει τη χρήση της βέλτιστης προσέγγισης για κάθε πόρο του συστήματος
- Μια κατά το δυνατόν βέλτιστη προσέγγιση περιλαμβάνει:
 - Διαμοίραση των πόρων σε ιεραρχικά διατεταγμένες κλάσεις
 - Χρήση των πλέον κατάλληλων τεχνικών για τη διαχείριση αδιεξόδων μέσα σε κάθε κλάση.

5. Πρόβλημα συνδαιτυμόνων φιλοσόφων

- Πέντε φιλόσοφοι κάθονται γύρω από ένα κυκλικό τραπέζι. Κάθε φιλόσοφος καταναλώνει το χρόνο του διαδοχικά σκεπτόμενος και τρώγοντας. Στο κέντρο του τραπεζιού υπάρχει ένα μεγάλο πιάτο με spaghetti. Κάθε φιλόσοφος χρειάζεται δύο πηρούνια για να φάει λίγο spaghetti.
- Υπάρχει ένα πηρούνι ανάμεσα σε κάθε ζεύγος φιλοσόφων και όλοι συμφωνούν ότι θα χρησιμοποιούν μόνον τα πηρούνια που βρίσκονται δεξιά και αριστερά από τον καθένα.

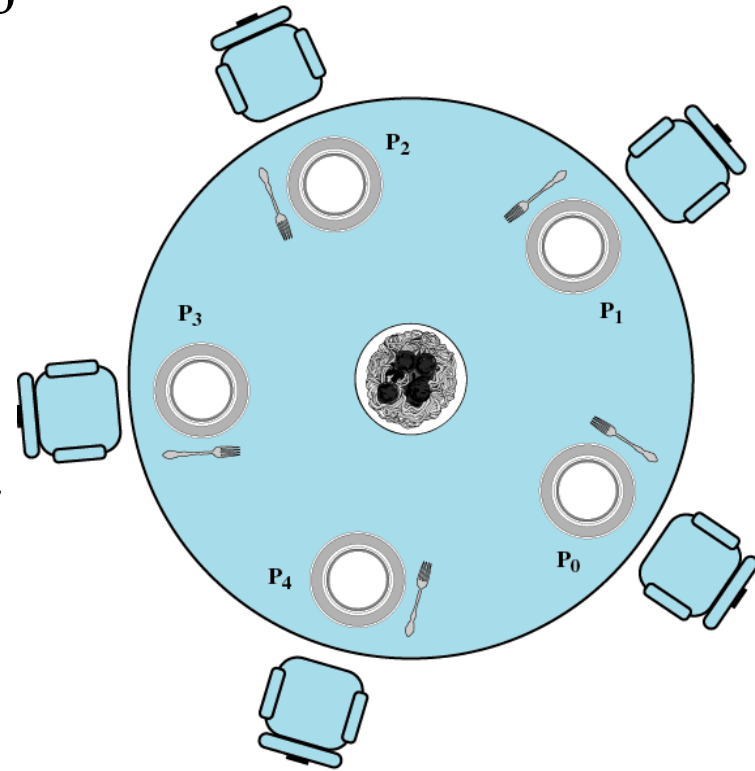


Figure 6.11 Dining Arrangement for Philosophers

Πρόβλημα συνδαιτυμόνων φιλοσόφων

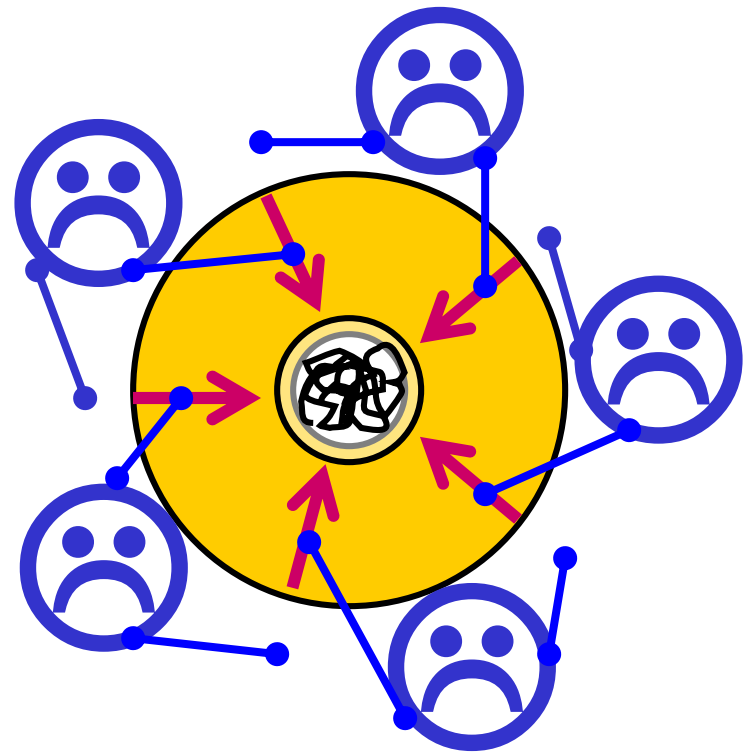
- Κάθε φιλόσοφος είναι μια διεργασία και κάθε πηρούνι είναι ένας διαμοιραζόμενος πόρος με ενέργειες δέσμευσης και απελευθέρωσης. Αν ένας φιλόσοφος πεινάσει, πρέπει πρώτα να πάρει τα πηρούνια δεξιά και αριστερά του δεξιό για να μπορέσει να ξεκινήσει να τρώει.
- Το πρόβλημα αυτό είναι ένα πρότυπο που χρησιμοποιείται για την αποτίμηση μεθόδων σχετικών με το συγχρονισμό ταυτόχρονων διεργασιών.
- Καταδεικνύει τη δυσκολία της εκχώρησης πόρων μεταξύ διεργασιών χωρίς αδιέξοδα και παρατεταμένες στέρσεις.
- Στόχος είναι η ανάπτυξη ενός πρωτοκόλλου απόκτησης των πηρουνιών που θα εξασφαλίζει
 - την απαλλαγή από αδιέξοδα
 - τη δικαιοσύνη: κανένας φιλόσοφος δεν πρέπει να υποφέρει από παρατεταμένη στέρηση
 - το μέγιστο δυνατό συγχρονισμό

Πρόβλημα συνδαιτυμόνων φιλοσόφων

- Δεν υπάρχει συμμετρική λύση
- Λύσεις
 - Προσθήκη ενός ακόμη πηρουνιού
 - Μέγιστος αριθμός 4 φιλοσόφων στο τραπέζι (κυκλική αναμονή)
 - Να εκτελείται διαφορετική αλληλουχία ενεργειών για τους φιλοσόφους με άρτιο και περιττό αύξοντα αριθμό δηλαδή δημιουργία δύο ομάδων φιλοσόφων. Η μία ομάδα (περιττός α/α) θα αποκτά πρώτα το δεξιό και μετά το αριστερό πηρούνι και η άλλη ομάδα (άρτιος α/α) πρώτα το αριστερό και μετά το δεξιό πηρούνι.
 - Ένας φιλόσοφος επιτρέπεται να αποκτήσει τα πηρούνια μόνον όταν και τα δύο είναι διαθέσιμα (κρίσιμο τμήμα) - (κατοχή και αναμονή)
 - Σχεδιασμός του συστήματος έτσι ώστε ένας φιλόσοφος να «κλέψει» ένα πηρούνι που δεν είναι γειτονικό του.

Πρόβλημα συνδαιτυμόνων φιλοσόφων

```
/* program diningphilosophers – deadlock and starvation */  
semaphore fork[5]={1};  
int i;  
void philosopher (int i)  
{  
    while (true)  
    {  
        think();  
        wait (fork[i]);  
        wait (fork[(i+1) mod 5]);  
        eat();  
        signal(fork[(i+1) mod 5]);  
        signal(fork[i]);  
    }  
}
```



Πρόβλημα συνδαιτυμόνων φιλοσόφων

```
/* program diningphilosophers – no deadlock – no starvation */  
semaphore fork[5]={1};  
semaphore room={4};  
int i;  
void philosopher (int i)  
{  
    while (true)  
    {  
        think();  
        wait(room);  
        wait (fork[i]);  
        wait (fork[(i+1) mod 5]);  
        eat();  
        signal(fork[(i+1) mod 5]);  
        signal(fork[i]);  
        signal(room);  
    }  
}
```

Πρόβλημα συνδαιτυμόνων φιλοσόφων

```
#define N 5 /* Number of philosophers */
#define RIGHT(i) (((i)+1) %N)
#define LEFT(i) (((i)==N) ? 0 : (i)+1)

typedef enum { THINKING, HUNGRY, EATING } phil_state;

phil_state state[N];
semaphore mutex =1;
semaphore s[N];      /* one per philosopher, all 0 */

void test(int i) {
    if ( state[i] == HUNGRY &&
        state[LEFT(i)] != EATING &&
        state[RIGHT(i)] != EATING ) {state[i] = EATING; V(s[i]);}
}

void get_forks(int i) {
    P(mutex);
    state[i] = HUNGRY;
    test(i);
    V(mutex);
    P(s[i]);
}
```

P -> wait
V -> signal

Πρόβλημα συνδαιτυμόνων φιλοσόφων

```
void put_forks(int i) {
    P(mutex);
    state[i] = THINKING;
    test(LEFT(i));
    test(RIGHT(i));
    V(mutex);
}

void philosopher(int process) {
    while(1) {
        think();
        get_forks(process);
        eat();
        put_forks(process);
    }
}
```